

LOW-LEVEL DESIGN

for

Identity Security Framework for Healthcare

SUDHEESH KS

Register No: 23BCCED034

Course: Bachelor of Computer Applications (BCA)

Yenepoya Institute of Arts, Science, Management & Commerce

Balmatta

Date: 16/04/2026

Under the guidance of

Ms. Divya N Shetty

Lecturer, Department of Computer Science

Yenepoya Institute of Arts, Science, Management & Commerce

Kulur

Submitted to



YENEPOYA

(DEEMED TO BE UNIVERSITY)

Recognized under Sec 3(A) of the UGC Act 1956

Accredited by NAAC with 'A' Grade

**YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND
MANAGEMENT**

Balmatta, Mangalore

Yenepoya (Deemed to be University)

Section No.	Title	Page No.
1	Introduction	1
1.1	Purpose of Document	2
1.2	Scope of the System	2
1.3	Definitions & Abbreviations	3
2	System Overview	3
2.1	System Description	4
2.2	Architecture Overview	4
2.3	Technology Stack	6
3	Module Design	7
4	Database Design	9
4.1	ER Diagram	10
4.2	Table Structures	10
4.3	Relationships	11
5	API Specifications	12
5.1	Authentication APIs	12
5.2	Authorization APIs	17
5.3	Logging APIs	19
5.4	Notification APIs	22
6	Security Considerations	24
6.1	Data Encryption	24
6.2	Authentication Mechanisms	25
6.3	Attack Prevention	27
7	Error Handling & Validation	28
8	Future Enhancements	29
9	Testing Strategy	29
9.1	Unit Testing	29

9.2	Integration Testing	30
10	Conclusion	30

1. Introduction

1.1 Purpose of Document

This document provides a detailed Low-Level Design (LLD) for the Identity Security Framework for Healthcare. It describes the internal architecture, module-level design, data flow, and component interactions required to implement a secure and scalable identity management system in a healthcare environment.

The purpose of this document is to:

- Define detailed system components and modules
- Describe authentication, authorization, and identity lifecycle flows
- Provide technical design for developers
- Ensure secure handling of sensitive healthcare data
- Document ML-based anomaly detection integration for security

This LLD acts as a blueprint for developers to implement the system efficiently while maintaining high security, compliance, and performance standards.

1.2 Scope of the System

The Identity Security Framework for Healthcare is designed to manage and secure digital identities of:

- Doctors
- Nurses
- Administrative Staff
- Patients

It provides:

- Secure user authentication (JWT + Session-based)
- Role-Based Access Control (RBAC)
- Multi-Factor Authentication (OTP via email)
- ML-based anomaly detection for fraud prevention
- Behavioral monitoring & threat detection
- Secure password lifecycle management
- Audit and logging of user activities

1.3 Definitions & Abbreviations

This section defines key terms and abbreviations used in the Identity Security Framework for Healthcare to ensure clarity and consistency throughout the document.

Definitions

- **Identity Management**
The process of managing user identities, including creation, authentication, authorization, and lifecycle management.
- **Authentication**
The process of verifying the identity of a user, including patients, employees, and administrators, by validating credentials such as passwords, OTPs (One-Time Passwords), or other authentication mechanisms.
- **Anomaly Detection**
Anomaly Detection is a security technique used to monitor and analyze the behavior of users, employees, and administrators to identify deviations from normal patterns, which may indicate potential security threats, fraud, or malicious attacks.
- **Behavioral Analysis**
Monitors activity patterns of users, employees, or administrators (such as login time, request frequency, and interaction behavior) to detect anomalies or suspicious actions in the system.
- **Session Management**
Manages sessions for administrators and patients (users) after authentication using session-based authentication. It ensures secure session creation, validation, and expiration. Maintains session integrity by preventing unauthorized access and session hijacking, and supports automatic session timeout and secure logout mechanisms to protect sensitive data.
- **Token-Based Authentication**
Employees use JSON Web Tokens (JWT) for authentication instead of session-based methods. A token is generated during employee login and is used to securely access protected APIs. The token carries user identity and permission data, ensuring stateless and scalable authentication across the system.
- **Multi-Factor Authentication (MFA)**
Multi-Factor Authentication (MFA) is implemented for employees during password setup and reset processes. It combines multiple verification

methods, including a secure token (email-based link) and OTP verification, to enhance security. This ensures validation of both possession and knowledge factors before granting access.

Abbreviations

Abbreviation	Full Form
JWT	JSON Web Token
RBAC	Role-Based Access Control
MFA	Multi-Factor Authentication
OTP	One-Time Password
ML	Machine Learning
AI	Artificial Intelligence
API	Application Programming Interface
DB	Database
SQL	Structured Query Language
XSS	Cross-Site Scripting
IP	Internet Protocol
UI	User Interface
UX	User Experience

2. System Overview

The Identity Security Framework for Healthcare is a secure hospital management system designed to manage patients, employees, and administrators with strong identity and access control. It supports key healthcare operations, including patient management, lab tests, prescriptions, and ward admissions. The system ensures secure, efficient handling of sensitive healthcare data while protecting against unauthorized access and cyber threats.

2.1 System Description

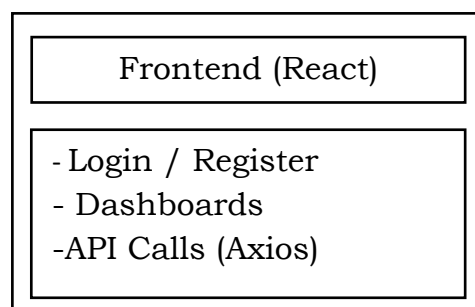
The system is implemented as a full-stack application using Node.js and Express.js for backend services, React for the frontend, and PostgreSQL for data storage, with communication handled through RESTful APIs. It uses session-based authentication for patients and administrators, and JWT-based authentication for employees.

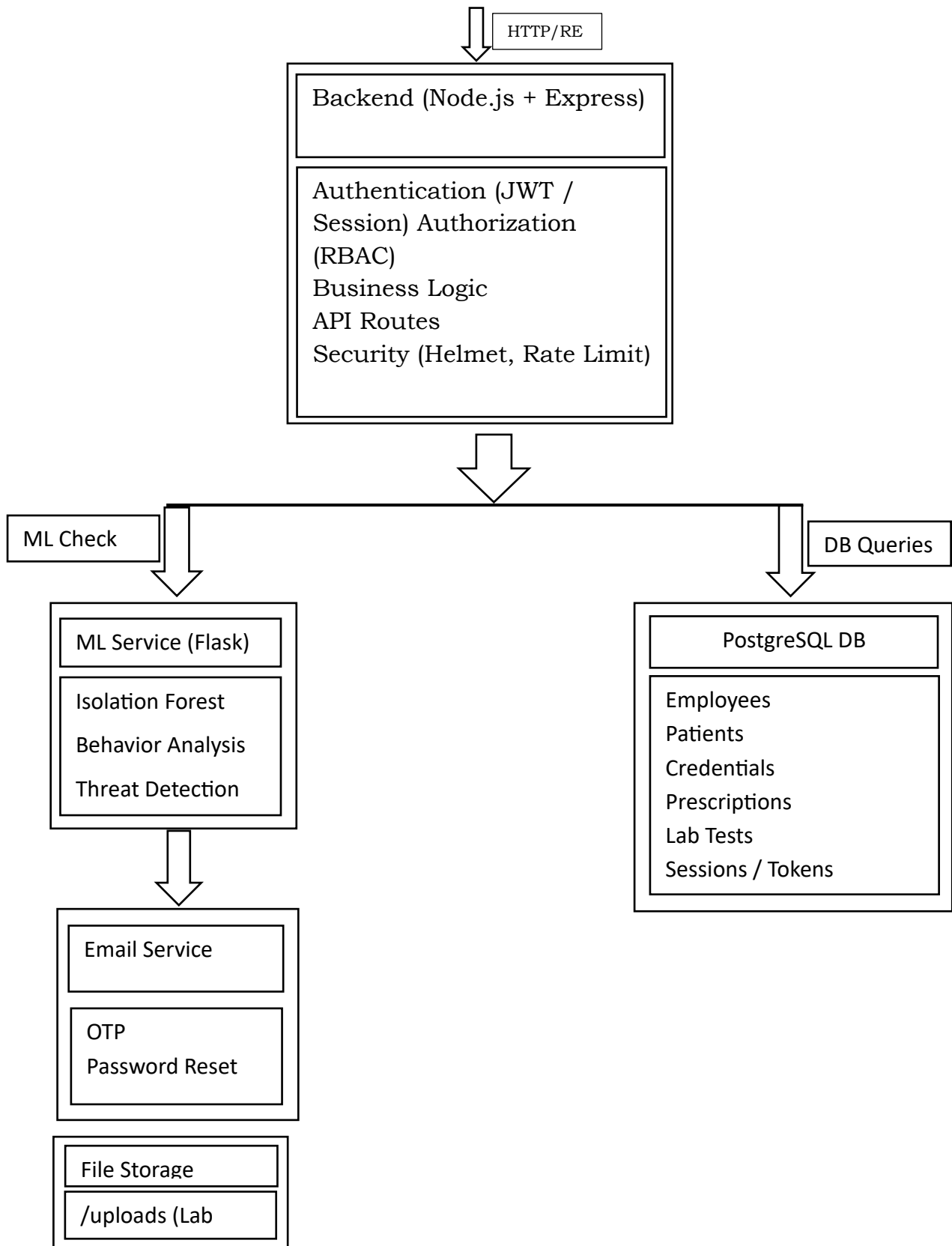
Advanced security features include OTP-based verification, role-based access control (RBAC), and machine learning-based anomaly detection via a Python Flask microservice. This service monitors user behavior and request patterns to detect suspicious activities. Additionally, all incoming requests are validated to prevent common attacks such as SQL injection, cross-site scripting (XSS), and automated bot threats, ensuring secure and reliable system operation.

2.2 Architecture Overview

The system follows a three-tier architecture to ensure scalability, maintainability, and security. This layered approach separates concerns, making the system easier to develop, test, and manage.

- **Presentation Layer (Frontend):** Built with React, it provides user interfaces for users, employees, and administrators, handling dashboards and API requests.
- **Application Layer (Backend):** Developed using Node.js and Express.js, it manages business logic, REST APIs, authentication (session-based for admins/patients and JWT for employees), and security features like OTP, rate limiting, and anomaly detection.
- **Data Layer (Database):** The system uses PostgreSQL as a relational database to securely store and manage structured data, including employee records, patient information, lab tests, prescriptions, credentials, and session data. Proper indexing, relationships, and constraints are applied to ensure data integrity, consistency, and efficient query performance.





2.3 Technology Stack

The system uses a modern and scalable technology stack across frontend, backend, database, and security layers.

Frontend

- React (Vite) – User interface development
- Axios – API communication
- CSS – Styling and layout

Backend

- Node.js – Server-side runtime
- Express.js – Web framework for APIs
- JWT (jsonwebtoken) – Token-based authentication (employees)
- Express-Session – Session-based authentication (admins & patients)
- Bcrypt – Password hashing
- Helmet – Security headers
- Express Rate Limit – API rate limiting
- Nodemailer – Email service (OTP, password reset)

Database

- PostgreSQL – Relational database management

Machine Learning Service

- Python (Flask) – ML API service
- Scikit-learn – Isolation Forest (anomaly detection)
- NumPy – Numerical computations
- Joblib – Model saving/loading

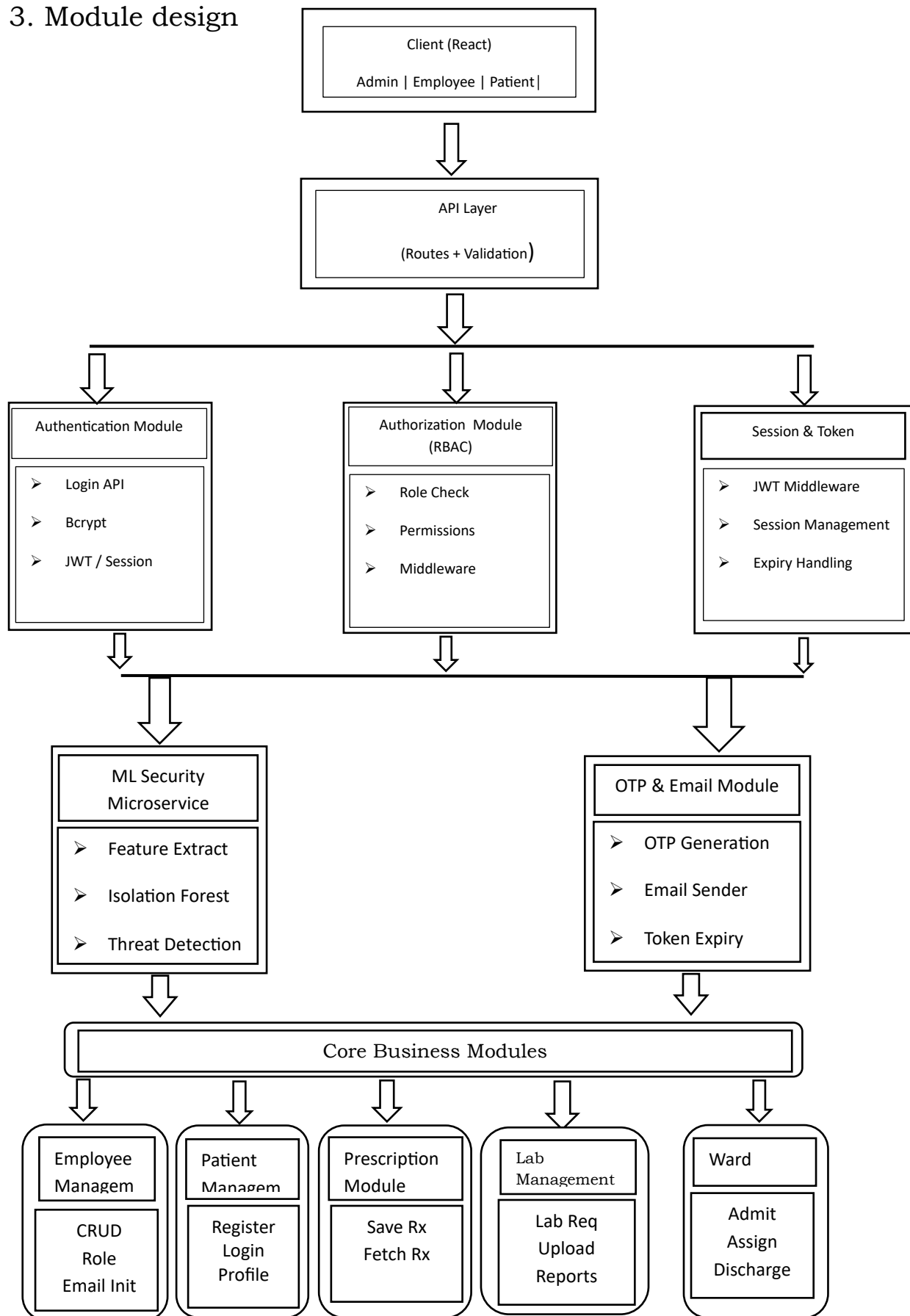
File Handling

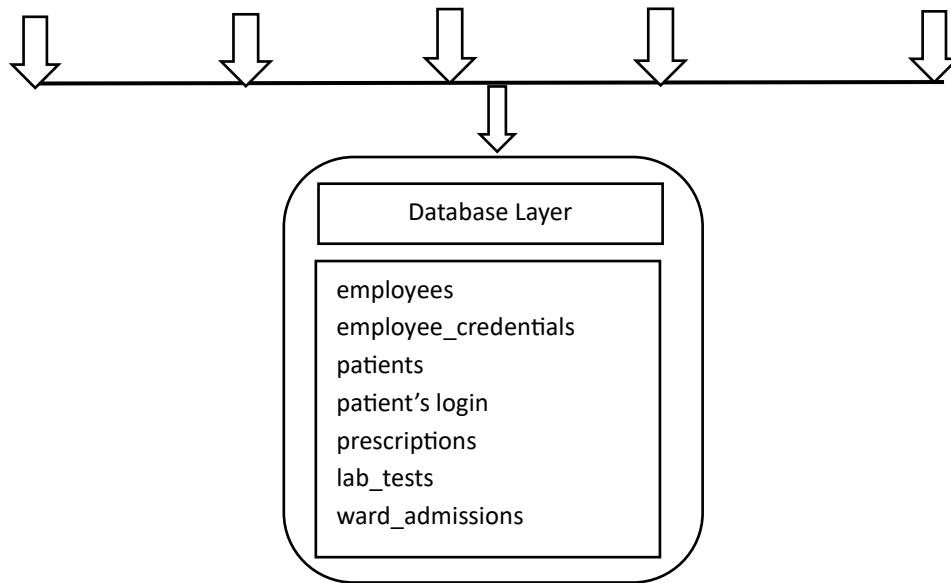
- Multer – File uploads (lab reports)
- Local Storage (/uploads) – File storage

Development Tools

- dotenv – Environment configuration
- ESLint – Code quality and linting

3. Module design





1. Authentication Module

Handles login and identity verification for users, employees, and administrators using password, session, or JWT.

2. Authorization Module

Implements role-based access control (RBAC) to restrict system access based on user roles and permissions.

3. Employee Management Module

Manages employee data including creation, update, deletion, credentials, and role assignments.

4. Patient Management Module

Handles patient registration, login, profile management, and appointment-related data.

5. ML Security Module

Analyzes request data and user behavior to detect anomalies and block suspicious activities.

6. OTP & Email Module

Generates and sends OTPs and secure links for password setup, reset, and verification.

7. Prescription Module

Stores and retrieves patient prescriptions, including medicines and notes.

8. Lab Management Module

Manages lab test requests, report uploads, and result tracking.

9. Ward Management Module

Handles patient admission, ward allocation, and discharge processes.

10. Session & Token Module

Manages session-based authentication for admins/patients and JWT-based authentication for employees.

11. API Layer

Provides RESTful endpoints, handles request validation, and connects frontend with backend services.

12. Database Module

Stores all system data including users, employees, patients, credentials, lab data, and logs.

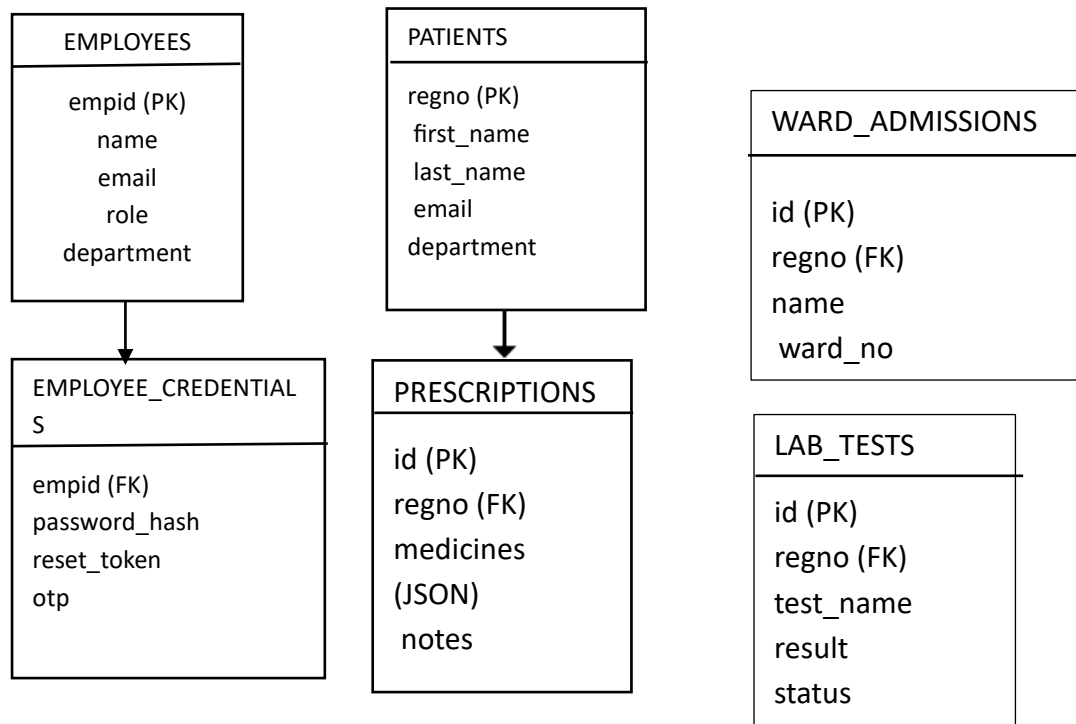
4 Database Design

The database is structured in a relational format using PostgreSQL to store all healthcare-related and security data.

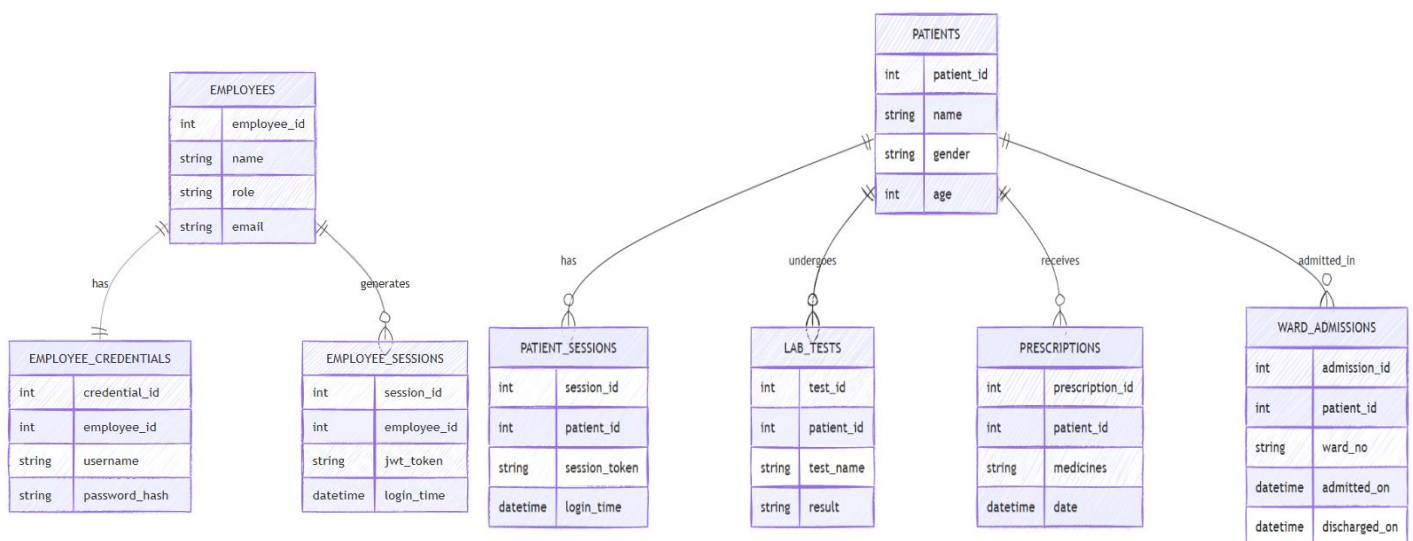
- Employees Table stores employee details such as empid, name, role, and department.
- Employee_Credentials Table stores authentication data including hashed passwords, OTP, and reset tokens linked to employees.
- Patients Table stores patient registration details and personal information.
- Prescriptions Table stores medicines and doctor notes in JSON format linked to patients.
- Lab_Tests Table stores test requests, results, and status tracking.
- Ward_Admissions Table manages patient admission details, ward allocation, and discharge records.

4.2 ER Diagram

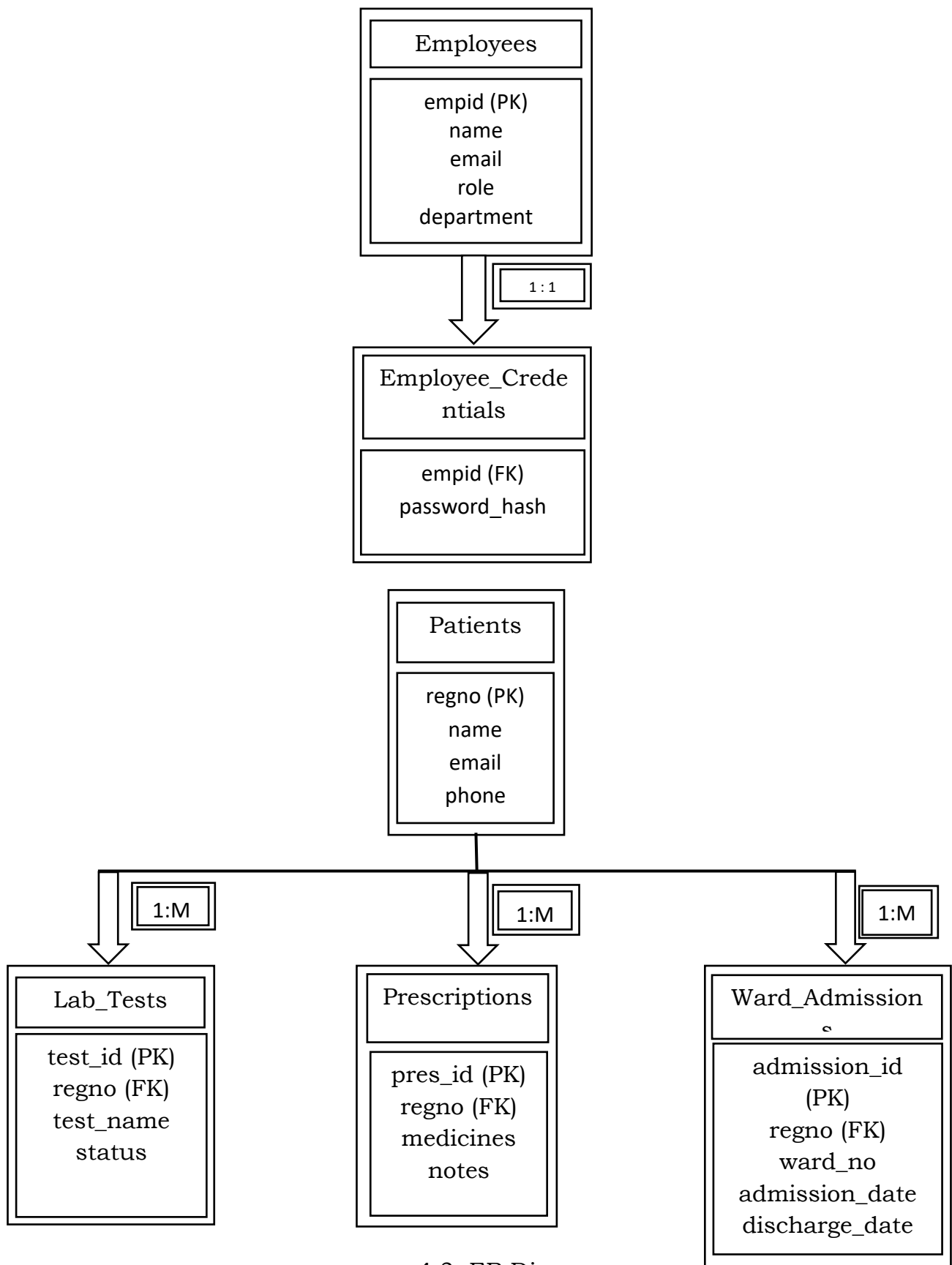
The database for the **Identity Security Framework for Healthcare** is designed using PostgreSQL and follows a relational structure. It manages users, employees, patients, medical records, authentication data, and security logs.



4.1 Table Structures



4 Relationships



4.3: ER Diagram

5 API Specifications

The system provides a set of RESTful APIs to manage authentication, healthcare operations, and security monitoring. All APIs are designed to be secure, scalable, and role-based, supporting patients (users), employees, and administrators.

5.2 Authentication APIs

The system implements a hybrid authentication model using JWT-based authentication for employees and session-based authentication for administrators and patients. All authentication flows are protected with security measures such as OTP verification, ML-based anomaly detection, and rate limiting.

Employee Authentication APIs (JWT-Based)

1. Employee Login

POST /api/employee/login

Authenticates an employee using employee ID and password.

Request Body:

```
{
  "empid": "E101",
  "password": "*****"
}
```

Response:

```
{
  "token": "jwt_token",
  "role": "nurse",
  "permissions": ["read_patients"],
  "message": "Login successful"
}
```

Status Codes:

- 200 – Login successful
- 401 – Invalid credentials

- 403 – Blocked (ML anomaly detected)

2. Get Logged-in Employee Profile

GET /api/employee/me

Headers:

Authorization: Bearer <JWT>

Description:

Returns the profile details of the authenticated employee.

Response:

```
{
  "empid": "E101",
  "name": "John Doe",
  "role": "nurse",
  "department": "General"
}
```

Status Codes:

- 200 – Success
- 401 – Unauthorized

3. Verify Password Reset Token

GET /api/employee/verify-token/:token

Description:

Validates the password reset token and checks expiration.

Response:

```
{
  "valid": true,
  "message": "Token is valid"
}
```

Status Codes:

- 200 – Token valid
- 400 – Invalid/expired token

4. Send OTP

POST /api/employee/send-otp

Request Body:

```
{
  "email": "employee@email.com"
}
```

Description:

Sends OTP to registered employee email for MFA.

Response:

```
{
  "message": "OTP sent successfully"
}
```

Status Codes:

- 200 – OTP sent
- 404 – Email not found

5. Change Password (Token + OTP)

POST /api/employee/change-password-token

Request Body:

```
{
  "token": "reset_token",
  "otp": "123456",
  "newPassword": "new_password"
}
```

Description:

Verifies token and OTP, then updates password securely.

Response:

```
{  
  "message": "Password updated successfully"  
}
```

Status Codes:

- 200 – Success
- 400 – Invalid OTP/token

Admin Authentication APIs (Session-Based)

6. Admin Login

POST /api/admin/login

Request Body:

```
{  
  "username": "admin",  
  "password": "*****"  
}
```

Description:

Authenticates admin using session-based login.

Response:

```
{  
  "message": "Login successful"  
}
```

Status Codes:

- 200 – Success
- 401 – Invalid credentials
- 403 – Blocked (ML detection)

Patient Authentication APIs (Session-Based)

7. Patient Login

POST /api/patient/login

Request Body:

```
{  
  "email": "patient@email.com",
```

```
    "password": "*****"  
  }
```

Response:

```
{  
  "message": "Login successful"  
}
```

Status Codes:

- 200 – Success
- 401 – Invalid credentials

8. Patient Session Check

GET /api/patient/session

Description:

Checks if patient session is active.

Response:

```
{  
  "loggedIn": true,  
  "patient": {  
    "regno": "P101",  
    "name": "Patient Name"  
  }  
}
```

Status Codes:

- 200 – Success
- 401 – Session not found

9. Patient Logout

POST /api/patient/logout

Description:

Destroys active session and logs out user.

Response:

```
{
  "message": "Logout successful"
}
```

Status Codes:

- 200 – Logout successful
- 401 – No active session

5.1 Authorization APIs

Authorization (RBAC – Role Based Access Control)

The system implements Role-Based Access Control (RBAC) to ensure that users can only access resources permitted to their role. Authorization is enforced using JWT verification (for employees) and session validation (for admins and patients).

1. Role-Protected Employee APIs (JWT-Based)

Get Patients (Nurse Only)

GET /api/patients/:empID

Headers:

Authorization: Bearer <JWT>

Description:

Fetches patient records. Accessible only by nurses.

Response:

```
{
  "patients": []
}
```

Status Codes:

- 200 – Success
- 403 – Forbidden
- 401 – Unauthorized

Lab Requests (Lab Staff Only)

GET /api/lab/requests/:empID

Headers:

Authorization: Bearer <JWT>

Description:

Retrieves lab test requests accessible only to authorized lab staff.

Response:

```
{  
  "requests": []  
}
```

Status Codes:

- 200 – Success
- 403 – Forbidden
- 401 – Unauthorized

2. Authorization Middleware (Core Security Layer)

Token Verification Middleware

verifyToken

Description:

- Validates JWT token
- Extracts user identity and role

Blocks request if token is invalid or expired

Role-Based Access Middleware

Headers:

authorizeRole(["nurse", "lab"])

Description:

- Checks user role

- Grants/denies access
- Returns 403 if unauthorized

3. Employee Profile Access Control

Get Employee Profile

GET /api/employee/me

- Requires valid JWT authentication
- Only authenticated employees can access their profile
- Returns employee details (role, department, permissions)

4. Admin Authorization (Session-Based)

Admin access is controlled using express-session.

Protected routes:

- /api/admin/login
- /api/admin/dashboard

Access Control:

- Active session required
- Unauthorized users blocked

5. Patient Authorization (Session-Based)

Patients use session-based authentication.

Protected routes:

- /api/patient/session
- /api/patient/profile

Access Control:

- Session must exist in server memory/database
- Valid session required
- Access denied if session invalid

5.3 Logging APIs

The system implements a centralized logging mechanism to track authentication events, security threats, system errors, and operational activities. These logs help in monitoring system behavior, detecting attacks, and maintaining audit trails.

1. Authentication & Security Logs

Admin Login Logging

POST /api/admin/login

Logs the following events:

- Successful admin login
- Failed login attempts
- IP-based blocking events
- Brute force detection using login attempt tracking

Employee Login Logging

POST /api/employee/login

Logs:

- Invalid employee ID attempts
- Incorrect password entries
- ML-based anomaly detection alerts
- Successful login resets attempt counter

Patient Login Logging

POST /api/patient/login

Logs:

- User not found errors
- Incorrect password attempts
- Suspicious login behavior (ML detection)
- Session creation events

2. ML-Based Security Logs

Anomaly Detection Logging

Function: checkAnomaly(requestData)

The ML service logs:

- Suspicious login patterns
- Bot-like behavior detection

- SQL Injection attempts
- XSS attack patterns
- ML service failure or timeout events

3. Email & OTP Logging

Email Service Logs (Nodemailer)

Tracked events:

- Employee registration email sent
- Password reset email sent
- OTP generation email sent

Logs include:

- Email delivery success
- Email delivery failure
- OTP expiration events

5. Lab & System Activity Logs

Lab Management Logs

- Lab test creation logs
- Lab result updates
- File upload tracking (reports/results)

Ward Management Logs

- Patient admission logs
- Patient discharge logs
- Ward updates and transfers

3. Security Event Logs

The system continuously tracks security-related events such as:

- IP blocking due to brute-force attacks
- Rate limit exceeded warnings
- Invalid JWT token usage
- Unauthorized role access attempts

Uses:

- express-rate-limit (rate control)
- JWT verification middleware
- IP tracking system

5.4 Notification APIs

The system implements a notification mechanism primarily based on email services and system-level alerts. Notifications are used to inform employees, patients, and administrators about account activities, security events, and hospital operations. The system uses Nodemailer for email delivery and backend triggers for internal status notifications.

1. Employee Account Notifications

Create Employee Notification

POST /api/employees

Description:

After successful employee creation, the system sends an email notification containing:

- Employee ID
- Temporary password
- Password reset link

Implementation:

- Implemented using Nodemailer
- Ensures secure onboarding of new employees

2. Password Reset & OTP Notifications

Verify Reset Token

GET /api/employee/verify-token/:token

Description:

- Validates password reset link sent via email
- Ensures token is not expired or reused

Send OTP

POST /api/employee/send-otp

Description:

Sends OTP to employee email for MFA verification.

Notification includes:

- OTP code
- Expiry time (5 minutes)
- Security warning for unauthorized attempts

3. Patient Registration Notifications

Patient Register

POST /api/patient/register

Description:

After successful registration, an email is sent containing:

- Registration ID
- Patient details
- Confirmation of account creation
- Appointment or next-step information (if applicable)

4. Lab Notification System (System-Based Alerts)

Lab Test Update APIs:

- PUT /api/lab/complete/:id
- PUT /api/lab/upload/:id

Description:

- Updates lab test status
- Stores test results
- Reflects completion status in the system

5. Ward & Admission Notifications

Patient Admission

POST /api/admit

Description:

- Stores admission details in the database
- Can be extended to send email alerts

Patient Discharge

PUT /api/discharge/:id

Description:

- Updates discharge status
- Triggers system-level notification update ID

6 Security Considerations

The system ensures strong security through multiple layers to protect data of users, employees, and administrators by using JWT-based authentication for employees and session-based authentication for admins and patients, along with OTP-based multi-factor authentication for sensitive operations. Role-Based Access Control (RBAC) restricts access based on user roles, while bcrypt hashing secures passwords and token/session expiry prevents misuse. Additional protections such as rate limiting and secure HTTP headers defend against common attacks, and a machine learning-based anomaly detection system continuously monitors user behavior to detect and block suspicious activities like brute force attacks, SQL injection, and bot activity, ensuring overall system safety and reliability.

6.1 Data Encryption

The system implements multiple layers of data encryption and secure authentication mechanisms to protect sensitive healthcare information such as user credentials, sessions, tokens, and OTPs. Security is enforced at both application level and transport level.

1. Password Encryption (Hashing)

Technology Used:

- bcrypt

Password Hashing:

```
const hashedPassword = await bcrypt.hash(password, 10);
```

Password Verification:

```
await bcrypt.compare(password, user.password_hash)
```

Purpose:

- Prevents exposure of plain-text passwords
- Secures data in case of database breach
- Ensures one-way encryption of sensitive credentials

2. Token Security (JWT Authentication)

Technology Used:

- JSON Web Token (JWT)

After successful employee login, a signed token is generated:

```
jwt.sign(payload, JWT_SECRET)
```

Security Features:

- Digitally signed tokens (tamper-proof)
- Time-limited expiration
- Encoded user identity and role
- Required for accessing protected APIs

3. Session Security (Admins & Patients)

Technology Used:

- express-session

Features:

- Secure session storage for admin and patient logins
- Session cookies include:
 - ✓ HTTP-only flag (prevents JavaScript access)
 - ✓ Secure flag (enabled in production)
 - ✓ Expiration time for automatic logout

6.2 Authentication Mechanisms

The system implements a hybrid authentication architecture combining JWT-based authentication, session-based authentication, Multi-Factor Authentication (MFA), Role-Based Access Control (RBAC), and ML-based security validation to ensure strong identity verification for all users.

1. Token-Based Authentication (Employees – JWT)

The employee authentication system is based on JSON Web Token (JWT).

Flow:

- Employee submits login credentials
- Backend validates credentials

- JWT token is generated and signed
- Token is sent to the client
- Token is used for all protected API requests via middleware

2. Session-Based Authentication (Admin)

Admin authentication is handled using express-session.

Flow:

- Admin logs in with credentials
- Server creates a session
- Session ID is stored in HTTP-only cookie
- Session is validated on each request

3. Session-Based Authentication (Patients)

Patients also use session-based authentication similar to admins.

4. Multi-Factor Authentication (MFA)

MFA is implemented for employees during sensitive operations such as password reset.

Components:

- Email-based reset token link
- OTP verification system

Flow:

- Password reset link sent to registered email
- OTP generated and sent separately
- User verifies both token and OTP
- Password is updated securely

5. Role-Based Authentication (RBAC)

RBAC is implemented using middleware:

`authorizeRole(["nurse", "doctor", "lab"])`

6. ML-Based Authentication Security

The system integrates a Machine Learning-based anomaly detection service for enhanced security.

Detects:

- Bot activity
- Suspicious login patterns
- Brute-force attacks
- SQL injection / XSS attempts

4 Attack Prevention

The system implements a multi-layered security architecture to protect against brute force attacks, injection attacks, unauthorized access, and automated bot activities. Each layer works together to ensure secure authentication and safe API usage across all modules.

Brute force attacks are prevented using an IP-based login attempt tracking mechanism. Every failed login attempt is recorded in a server-side in-memory store, and repeated failures from the same IP result in temporary blocking. This dynamic blocking strategy helps reduce credential guessing attacks.

To protect against high-frequency abuse and denial-of-service attempts, the system uses rate limiting through express-rate-limit. This restricts the number of requests allowed per IP within a defined time window, ensuring system stability under heavy or malicious traffic.

An ML-based security layer is integrated to detect advanced attack patterns that traditional rules cannot identify. The machine learning service analyzes incoming requests for SQL injection patterns, XSS attempts, path traversal behavior, bot-like activity, and abnormal login patterns. If the anomaly score exceeds the defined threshold, the request is immediately blocked with a 403 response.

Authentication security is enforced using JWT and session-based mechanisms. JWT tokens are signed, time-limited, and tamper-proof, ensuring secure access to employee APIs. Admin and patient sessions are managed using secure server-side sessions with HTTP-only cookies to prevent client-side tampering.

Password security is ensured using bcrypt hashing with salt, preventing storage of plain-text passwords and protecting against database leaks. Even if credentials are exposed, they cannot be reversed.

Additional protection is provided through security headers using Helmet, which defends against XSS attacks, clickjacking, and MIME-type sniffing

vulnerabilities. This ensures that API responses are delivered with hardened HTTP security configurations.

Finally, OTPs and password reset tokens are time-bound. OTPs expire within 5 minutes and reset tokens expire within 10 minutes, preventing replay attacks and unauthorized reuse of authentication links.

7 Error Handling & Validation

The system implements a multi-layer error handling and validation strategy to ensure data integrity, secure execution, and stable API behavior across all modules.

At the request level, every API performs strict input validation before processing any business logic. Required fields are checked for completeness and correctness to prevent invalid or incomplete data from entering the system. For example, employee creation requires fields such as name, email, role, and shift timings, while patient registration requires mandatory identity and contact details. This prevents malformed requests and ensures consistent data quality.

Database-level errors are handled using PostgreSQL error codes. Common constraints such as duplicate entries are managed using specific error identifiers like 23505 for unique constraint violations (e.g., duplicate email). Instead of exposing raw database errors to the client, the system returns controlled and user-friendly messages, improving security and user experience.

All backend APIs are wrapped in global try-catch blocks to ensure runtime stability. Any unexpected error is logged internally for debugging purposes while the client receives a safe, generic error response. This prevents system crashes and maintains continuous service availability.

A machine learning-based validation layer is also integrated into the request pipeline. Every critical request is analyzed by an external ML service to detect anomalies such as SQL injection patterns, XSS attempts, bot-like behavior, and unusual login activity. If the anomaly score exceeds the defined threshold, the request is blocked immediately with a 403 Forbidden response.

Authentication-related errors are handled separately for both JWT and session-based systems. Invalid or tampered JWT tokens result in a 401 Unauthorized response, while expired tokens are rejected with appropriate error messages. For session-based authentication, missing or invalid sessions lead to access denial, ensuring that only authenticated users can access protected resources.

Email and OTP validation is enforced during sensitive operations such as password reset and multi-factor authentication. OTP values are verified against stored records and checked for expiration before allowing further actions. Similarly, reset tokens are validated before password updates to prevent unauthorized access.

Finally, business logic validation ensures domain-level correctness. Employees are restricted to valid shift hours during login, patient records require unique email identifiers, and lab test operations require valid registration numbers. These validations enforce system rules beyond basic input checks, maintaining overall consistency and reliability.

8 Future Enhancements

The system can be further improved by adding advanced features to enhance security, scalability, and usability. Future enhancements may include integration of biometric authentication such as fingerprint or facial recognition for stronger identity verification. A real-time notification system (SMS or push notifications) can be implemented to provide instant alerts for appointments, lab results, and security events.

The ML-based anomaly detection module can be improved by training with larger real-world datasets to reduce false positives and increase accuracy. Additionally, implementing a centralized logging and monitoring system (such as ELK stack) would help in better analysis of system activities and security incidents.

The system can also be scaled using cloud deployment and microservices architecture to handle a larger number of users and hospital branches. Mobile application support for patients and employees can improve accessibility and user experience.

Finally, advanced role management, audit trails, and compliance features (such as healthcare data regulations) can be added to make the system more robust and suitable for enterprise-level healthcare environments.

9 Testing Strategy

The system follows a structured testing approach to ensure reliability, security, and correctness of all modules.

9.1 Unit Testing

Unit testing is performed to validate individual components and functions in isolation. Each module, such as authentication, API services, utility functions, and data processing logic, is tested independently to ensure correct functionality. For example, validation functions, JWT token generation,

password hashing, and helper utilities are tested with different input cases. This helps in identifying bugs at an early stage and ensures that each unit behaves as expected.

9.2 Integration Testing

Integration testing is used to verify the interaction between different system components. It ensures that the frontend, backend APIs, database, and ML security service work together correctly. For instance, login functionality is tested end-to-end, including authentication, session/JWT handling, database validation, and ML anomaly checks. This testing confirms that data flows correctly across modules and that combined functionalities operate without errors.

10. Conclusion

The Identity Security Framework for Healthcare is a secure and scalable hospital management system designed to manage users, employees, and administrators with strong data protection.

It uses JWT for employee authentication, session-based login for patients and admins, and OTP-based verification for sensitive actions. A machine learning-based anomaly detection system adds an extra layer of security by identifying suspicious activities like brute force attacks, injection attempts, and bot behavior.

Built with Node.js, Express.js, and PostgreSQL, the system ensures reliable data management for hospital operations such as patients, lab tests, prescriptions, and wards. Overall, it provides a secure, efficient, and modern solution for healthcare identity and access management.